

RideLink Account Service

Setup, Run, Postman Testing & Verification Guide

IT3130 - Application Development

Account Service

Owner: Fernando B S C

Student ID: IT24103775

Verification date: 29 September 2026

Table of Contents

Table of Contents	2
1. Document Purpose	3
2. Scope and Ownership	3
3. Verified Technology Stack	3
4. Project Location and Structure	3
5. Prerequisites	4
6. Secure Environment Configuration	4
7. MongoDB Atlas Setup	4
8. Run in Visual Studio Code	4
9. Build and Automated Tests	5
10. Startup Verification	5
11. Base URL and Swagger	5
12. Postman Setup	6
13. Complete Endpoint Reference	6
14. Passenger Registration	6
15. Driver Registration	7
16. Duplicate Email Validation	7
17. Registration Validation Matrix	7
18. Login and JWT	8
19. Login Failure Tests	8
20. Profile Retrieval	8
21. Profile Update	9
22. Account Status Management	9
23. Role-Based Authorization Matrix	9
24. JWT Negative Testing	10
25. Password Security	10
26. MongoDB Persistence Verification	10
27. Complete QA Verification Matrix	11
28. Troubleshooting	11
29. Security Notes	12
30. Final Verification Summary	12
31. Viva Quick Reference	12
Appendix A. PowerShell Smoke-Test Checklist	13
Appendix B. Evidence Classification	13

1. Document Purpose

This guide explains how to configure, run, and test the implemented RideLink Account Service from Visual Studio Code, Swagger UI, and Postman. It is based on direct inspection of the repository and the automated verification performed on 29 September 2026. Results use PASS, FAIL, PARTIAL, BLOCKED, or NOT APPLICABLE only.

Important evidence boundary: the Maven build and 23 automated tests were executed successfully. A live runtime attempt was also executed, but startup correctly stopped because the required `JWT_SECRET` and `MONGODB_URI` were not present in the execution environment. Therefore all Atlas-backed HTTP, persistence, Swagger, and Postman runtime checks are marked BLOCKED or PARTIAL rather than being invented.

2. Scope and Ownership

The Account Service owns authentication credentials, passenger and driver identities, roles, profile information, and account status. Driver vehicles, availability, service area, location, rides, fares, and payments are outside this guide.

Database ownership rule: Account Service alone owns its MongoDB database. Driver Service, Ride Service, and Fare & Payment Service must not query it directly. They exchange stable account IDs and use API contracts.

CRUD classification:

- CREATE: passenger registration and driver identity registration.
- READ: account/profile retrieval.
- UPDATE: account/profile update and account-status update.
- DELETE ACCOUNT: Not implemented / not required by the current Account Service specification.

3. Verified Technology Stack

Item	Actual repository value
Language	Java 17
Framework	Spring Boot 3.3.4
Build	Maven multi-module project; no Maven wrapper
Package	<code>com.ridelink.account</code>
Persistence	Spring Data MongoDB / MongoDB Atlas
Database	<code>ridelink_account_db</code> by default
Collection	<code>accounts</code>
Security	Spring Security, stateless JWT, BCrypt
JWT library	JJWT 0.12.6
API docs	Springdoc OpenAPI 2.5.0
Tests	JUnit 5, Mockito, MockMvc, Spring Security Test

4. Project Location and Structure

Account Service directory: `account-service`

Key implementation areas:

```
account-service/
  pom.xml
  src/main/java/com/ridelink/account/
    config/      Mongo, OpenAPI, and security configuration
    controller/  Authentication and account REST APIs
    document/    MongoDB Account document
    domain/      Role and AccountStatus enums
    dto/         Validated request and safe response records
    exception/   Centralized JSON error handling
    repository/  Spring Data MongoDB repository
    security/    JWT service/filter and authorization helpers
    service/     Registration, login, profile, and status rules
  src/main/resources/application.yml
  src/test/java/com/ridelink/account/
```

5. Prerequisites

1. Windows with Visual Studio Code.
2. JDK 17. Verify that `JAVA_HOME` points to a JDK 17 installation.
3. Apache Maven 3.9 or compatible. The repository has no `mvnw.cmd`.
4. A MongoDB Atlas cluster and a database user with access to the Account Service database.
5. The developer machine IP allowed in the Atlas Network Access list.
6. Postman for the shared collection, or a browser for Swagger UI.

6. Secure Environment Configuration

The application requires both `MONGODB_URI` and `JWT_SECRET`. It refuses to start when a required secret is absent. The database and expiry values have safe defaults.

Open a new PowerShell terminal in VS Code and set placeholders locally:

```
$env:MONGODB_URI="<YOUR_MONGODB_ATLAS_CONNECTION_STRING>"
$env:MONGODB_DATABASE="ridelink_account_db"
$env:JWT_SECRET="<YOUR_SECURE_JWT_SECRET>"
$env:JWT_EXPIRATION_MS="3600000"
$env:ACCOUNT_SERVICE_PORT="8081"
```

`JWT_SECRET` must contain at least 32 bytes. Never commit a real URI or secret. Do not place credentials in Postman, source code, screenshots, or this PDF.

7. MongoDB Atlas Setup

1. Create or select an Atlas cluster.
2. Create a database user with only the permissions required by the Account Service.
3. Add the development machine's current IP to Atlas Network Access. Avoid an unrestricted production allow-list.
4. Copy the driver connection string and set it only in the local `MONGODB_URI` environment variable.
5. Keep `MONGODB_DATABASE=ridelink_account_db` unless the team intentionally supplies another isolated Account database.
6. Start the service and confirm that the unique `email` index is created on the `accounts` collection.

The URI, username, password, hostname, JWT secret, JWT values, and password hashes are intentionally excluded from this guide.

8. Run in Visual Studio Code

1. Open `D:\IT3130_AD_Group_Assignment_RideLink` in VS Code.
2. Select **Terminal -> New Terminal**.
3. Run these copy-ready PowerShell commands:

```
cd "D:\IT3130_AD_Group_Assignment_RideLink\account-service"
java -version
mvn -version
mvn clean test
```

```
$env:MONGODB_URI="<YOUR_MONGODB_ATLAS_CONNECTION_STRING>"
$env:MONGODB_DATABASE="ridelink_account_db"
$env:JWT_SECRET="<YOUR_SECURE_JWT_SECRET>"
$env:JWT_EXPIRATION_MS="3600000"
$env:ACCOUNT_SERVICE_PORT="8081"
```

```
mvn spring-boot:run
```

Successful startup evidence should include Tomcat listening on port 8081 and **Started AccountServiceApplication**. Keep the terminal running while using Swagger or Postman.

9. Build and Automated Tests

From the repository root:

```
mvn -pl account-service clean test
mvn -pl account-service clean package
```

Actual result on 29 September 2026:

Check	Result	Evidence
Compilation	PASS	42 main source files and 5 test source files compiled with Java 17
Automated tests	PASS	23 run, 23 passed, 0 failed, 0 errors, 0 skipped
Package build	PASS	Executable Spring Boot JAR created
Runtime startup	BLOCKED	Required environment secrets were not supplied

10. Startup Verification

The packaged application was started without secrets to verify secure failure behavior. Tomcat selected the configured port 8081, then application context creation stopped because `#{JWT_SECRET}` could not be resolved. This is expected fail-closed behavior, not a successful runtime.

After setting both required secrets, verify:

- no bean-creation or dependency errors;
- **Started AccountServiceApplication** appears;
- the process remains running;
- Atlas accepts the connection and index creation;
- no URI, password, JWT secret, or complete token appears in logs.

11. Base URL and Swagger

Configured Account Service base URL:

```
http://localhost:8081
```

Configured documentation URLs:

```
Swagger UI: http://localhost:8081/swagger-ui/index.html
Alias:      http://localhost:8081/swagger-ui.html
OpenAPI JSON:http://localhost:8081/v3/api-docs
```

The paths are verified from `application.yml`, Spring Security rules, and Springdoc configuration. Live browser loading is **BLOCKED** until the service starts with valid environment configuration.

12. Postman Setup

Import:

- Collection: `postman/collections/RideLink.postman_collection.json`
- Environment: `postman/environments/RideLink-Local.postman_environment.json`

Account variables:

Variable	Safe value/purpose
<code>account_service_url</code>	<code>http://localhost:8081</code>
<code>token</code>	Automatically stores login JWT; never export a real value
<code>passenger_id</code>	Automatically stores passenger registration ID
<code>driver_id</code>	Automatically stores driver registration ID
<code>admin_token</code>	Not present; no verified admin bootstrap workflow exists

Select the environment, start the service, and execute the requests in the **01 Account Service** folder in order.

13. Complete Endpoint Reference

Method	Endpoint	Authentication	Role	Purpose	Success	Verification
POST	<code>/api/auth/register/passenger</code>	None	Public	Create passenger identity	201	PASS automated; live BLOCKED
POST	<code>/api/auth/register/driver</code>	None	Public	Create driver identity only	201	PASS automated; live BLOCKED
POST	<code>/api/auth/login</code>	None	Public	Validate credentials and issue JWT	200	PASS automated; live BLOCKED
GET	<code>/api/accounts/{id}</code>	Bearer JWT	Owner or ADMIN	Read safe profile	200	PARTIAL automated security
PUT	<code>/api/accounts/{id}</code>	Bearer JWT	Owner or ADMIN	Update name and phone	200	PASS service/API tests
PATCH	<code>/api/accounts/{id}/status</code>	Bearer JWT	ADMIN	Change account status	200	PASS RBAC tests; live admin BLOCKED

14. Passenger Registration

Name: Register Passenger

Method: POST

URL: `{{account_service_url}}/api/auth/register/passenger`

Authentication: none

Header: `Content-Type: application/json`

```
{
  "fullName": "Test Passenger",
  "email": "passenger01@example.com",
  "password": "Passenger@123",
  "phone": "+94712345678"
}
```

Expected 201 response shape:

```
{
  "id": "<generated-account-id>",
  "fullName": "Test Passenger",
  "email": "passenger01@example.com",
  "role": "PASSENGER",
  "phone": "+94712345678",
  "status": "ACTIVE",
  "createdAt": "<ISO-8601 timestamp>",
  "updatedAt": "<ISO-8601 timestamp>"
}
```

The Postman test script stores `id` as `passenger_id`. Automated tests verify role, status, email normalization, password hashing input, 201 behavior, and absence of `passwordHash`. Live Atlas record verification is BLOCKED.

15. Driver Registration

Name: Register Driver

Method: POST

URL: `{{account_service_url}}/api/auth/register/driver`

Authentication: none

Header: Content-Type: application/json

```
{
  "fullName": "Test Driver",
  "email": "driver01@example.com",
  "password": "Driver@123",
  "phone": "+94787654321"
}
```

Expected status is 201 with the same safe account fields and `role: DRIVER`, `status: ACTIVE`. Postman stores the returned ID as `driver_id`. This creates only the Driver Account identity; vehicles, availability, service area, and location belong to Driver & Vehicle Service.

16. Duplicate Email Validation

Repeat either registration with the same normalized email. Expected implementation behavior:

- HTTP 409 Conflict;
- `error` is `DUPLICATE_EMAIL`;
- message does not expose database internals;
- MongoDB unique index prevents concurrent duplicates.

```
{
  "timestamp": "<ISO-8601 timestamp>",
  "status": 409,
  "error": "DUPLICATE_EMAIL",
  "message": "An account with this email already exists",
  "path": "/api/auth/register/passenger",
  "fieldErrors": {}
}
```

Automated service verification: PASS. Live database duplicate check: BLOCKED.

17. Registration Validation Matrix

Test	Actual rule	Expected HTTP result	Verification
Blank full name	@NotBlank, max 100	400 VALIDATION_ERROR	Implemented; validation framework tested
Blank email	@NotBlank, @Email	400 VALIDATION_ERROR	Implemented
Invalid email	@Email, max 254	400 VALIDATION_ERROR	Implemented
Blank password	@NotBlank	400 VALIDATION_ERROR	Implemented
Short password	8-72 characters	400 VALIDATION_ERROR	Implemented
Blank phone	@NotBlank	400 VALIDATION_ERROR	Implemented
Invalid phone	optional +, digit first, 7-20 allowed characters	400 VALIDATION_ERROR	Implemented
Malformed JSON	readable JSON required	400 MALFORMED_REQUEST	Implemented

Example invalid body:

```
{"fullName":"","email":"not-an-email","password":"short","phone":"bad"}
```

18. Login and JWT

Method: POST

URL: `{{account_service_url}}/api/auth/login`

Authentication: none

Header: Content-Type: `application/json`

```
{
  "email": "passenger01@example.com",
  "password": "Passenger@123"
}
```

Expected 200 response shape:

```
{
  "token": "<REDACTED_JWT_TOKEN>",
  "tokenType": "Bearer",
  "expiresInMs": 3600000,
  "account": {
    "id": "<id>",
    "fullName": "Test Passenger",
    "email": "passenger01@example.com",
    "role": "PASSENGER",
    "phone": "+94712345678",
    "status": "ACTIVE",
    "createdAt": "<timestamp>",
    "updatedAt": "<timestamp>"
  }
}
```

The Postman script stores the token in `token`. Protected requests use `Authorization: Bearer {{token}}`. JWT claims are limited to subject/account ID, email, role, issue time, and expiry.

19. Login Failure Tests

Scenario	Actual expected status/code	Automated evidence
Wrong password	401 INVALID_CREDENTIALS	PASS
Unknown email	401 INVALID_CREDENTIALS	PASS
Missing email	400 VALIDATION_ERROR	Validation implemented
Missing password	400 VALIDATION_ERROR	Validation implemented
Malformed JSON	400 MALFORMED_REQUEST	Handler implemented
SUSPENDED account	403 ACCOUNT_SUSPENDED	PASS service test
DISABLED account	403 ACCOUNT_DISABLED	PASS service test

The same generic invalid-credentials message is used for unknown email and incorrect password to reduce account enumeration risk.

20. Profile Retrieval

Method: GET

URL: `{{account_service_url}}/api/accounts/{{passenger_id}}`

Header: `Authorization: Bearer {{token}}`

Required role: matching account owner or ADMIN

Expected 200 response is the safe `AccountResponse` shape shown in registration. It never includes password or password hash.

Negative checks:

- no Authorization header -> 401 `UNAUTHORIZED` (PASS API test);
- invalid/malformed bearer token -> 401 (PASS API test);
- authenticated user requesting another account -> 403 by owner authorization rule;
- missing account -> 404 `ACCOUNT_NOT_FOUND`.

21. Profile Update

Method: PUT

URL: `{{account_service_url}}/api/accounts/{{passenger_id}}`

Headers: Content-Type: application/json, Authorization: Bearer `{{token}}`

```
{
  "fullName": "Updated Passenger",
  "phone": "+94770000000"
}
```

Only `fullName` and `phone` exist in `UpdateAccountRequest`. Email, role, status, ID, timestamps, password, and password hash cannot be mass-assigned through this operation. Expected status is 200 with the updated safe profile. Service update and validation tests: PASS. Live read-after-write persistence: BLOCKED.

22. Account Status Management

Method: PATCH

URL: `{{account_service_url}}/api/accounts/{{passenger_id}}/status`

Headers: Content-Type: application/json, Authorization: Bearer `<ADMIN_TOKEN>`

Required role: ADMIN

```
{"status": "SUSPENDED"}
```

Allowed values are `ACTIVE`, `SUSPENDED`, and `DISABLED`. Passenger and driver attempts return 403; both are verified by API tests. An ADMIN mock-security test verifies the allowed controller path.

ADMIN TEST BLOCKED: no public admin registration or verified admin bootstrap mechanism exists. Do not use a passenger token for this request and do not fabricate an admin token. Live status transition, suspended/disabled login, and restoration to `ACTIVE` require a controlled test-admin bootstrap.

23. Role-Based Authorization Matrix

Operation	Public	Passenger	Driver	Admin
Register passenger	Allowed	Allowed	Allowed	Allowed
Register driver identity	Allowed	Allowed	Allowed	Allowed
Login	Allowed	Allowed	Allowed	Allowed
Get own profile	No	Allowed	Allowed	Allowed
Get another profile	No	Forbidden	Forbidden	Allowed
Update own profile	No	Allowed	Allowed	Allowed
Update another profile	No	Forbidden	Forbidden	Allowed
Update account status	No	Forbidden	Forbidden	Allowed

24. JWT Negative Testing

Test	Expected behavior	Verification
No header	401 JSON error	PASS MockMvc
Invalid bearer token	401	PASS MockMvc
Malformed JWT	401	PASS through invalid-token filter path
Expired JWT	Rejected during signature/expiry parsing	PASS unit test
Valid JWT for ACTIVE stored account	Security context established	PARTIAL; crypto claims tested
Token for suspended/disabled account	No authentication context	Implemented; live BLOCKED

Never paste a complete token into reports or screenshots. Use `<REDACTED_JWT_TOKEN>`.

25. Password Security

- `BCryptPasswordEncoder` hashes registration passwords before persistence.
- Login uses `matches()` rather than plaintext comparison.
- The Mongo document stores only `passwordHash`.
- Response DTOs contain no password or hash field.
- JWTs contain no password data.
- Tests assert `passwordHash` is absent from API JSON and JWT claims.
- Secrets, tokens, passwords, and hashes must not be logged.

26. MongoDB Persistence Verification

Item	Verified repository design	Live result
Database platform	MongoDB Atlas	BLOCKED without URI
Logical database	<code>ridelink_account_db</code>	Configuration verified
Collection	<code>accounts</code>	Mapping verified
Unique email	<code>@Indexed(unique=true)</code> plus service pre-check	Design PASS; live index BLOCKED
Created/updated timestamps	Spring Data auditing	Design PASS; live record BLOCKED
Passenger/driver persistence	<code>AccountRepository.save()</code>	Unit PASS; Atlas BLOCKED
Profile/status persistence	Repository save	Unit PASS; Atlas BLOCKED

Safe Atlas verification after startup: register fictional accounts, inspect only the safe fields in Atlas Data Explorer, confirm the unique email index, update the profile, and verify the same account ID changed. Never expose the stored hash.

27. Complete QA Verification Matrix

ID	Scenario	Expected	Actual evidence	Result
AS-01	Passenger registration	201, PASSENGER/ACTIVE	Service + API tests	PASS
AS-02	Driver registration	201, DRIVER/ACTIVE	Service test	PASS
AS-03	Duplicate email	409	Exception/service tests	PASS
AS-04	Missing name	400	Constraints inspected	PARTIAL
AS-05	Invalid email	400	Constraints inspected	PARTIAL
AS-06	Invalid password length	400	Constraints inspected	PARTIAL
AS-07	Valid login	200 with JWT	Service/API tests	PASS
AS-08	Invalid password	401	Service test	PASS
AS-09	Unknown account	401	Service test	PASS
AS-10	Get profile authenticated	200	Service test; live blocked	PARTIAL
AS-11	Get profile without token	401	MockMvc	PASS
AS-12	Update profile	200	Service test	PASS
AS-13	Invalid profile update	400	MockMvc	PASS
AS-14	Passenger status attempt	403	MockMvc	PASS
AS-15	Suspended login	403	Service test	PASS
AS-16	Disabled login	403	Service test	PASS
AS-17	passwordHash exposure	Absent	API/JWT tests	PASS
AS-18	MongoDB persistence	Record/index verified	Credentials unavailable	BLOCKED
AS-19	Swagger UI	Page loads	Runtime unavailable	BLOCKED
AS-20	OpenAPI document	JSON loads	Runtime unavailable	BLOCKED
AS-21	Maven tests	All pass	23/23 passed	PASS
AS-22	Package build	Success	BUILD SUCCESS	PASS

28. Troubleshooting

Problem	Likely cause / verification	Safe solution
JAVA_HOME invalid	mvn reports Java home error	Point JAVA_HOME to JDK 17; reopen terminal
mvn not recognized	Maven is not on PATH	Install Maven or use the IntelliJ-bundled Maven executable; no wrapper exists
JWT_SECRET placeholder error	Variable absent	Set a local secret of at least 32 bytes; do not commit it
MONGODB_URI missing	Required placeholder unresolved	Set the Atlas URI in the current terminal only
Atlas connection timeout	IP not allowed, DNS/firewall issue	Check Atlas Network Access and local connectivity
Atlas authentication failed	Wrong user/password or permissions	Rotate/check Atlas database user; update local URI only
Port 8081 in use	Another process owns the port	Stop that process or set ACCOUNT_SERVICE_PORT
Swagger 404	Wrong path or service not running	Use /swagger-ui/index.html; verify startup first
401 Unauthorized	Missing/invalid/expired token	Login again and set Authorization: Bearer {{token}}
403 Forbidden	Authenticated role/owner is not allowed	Use the correct owner identity or controlled ADMIN account
409 duplicate email	Normalized email already exists	Use a new fictional email or intentionally verify duplicate handling
Tests should not hit Atlas	Unit tests use mocks/slices	Never point automated tests at a production cluster

29. Security Notes

No real MongoDB URI, username, password, Atlas hostname, JWT secret, bearer token, or BCrypt hash appears in this document. Configuration placeholders are deliberate. Registration cannot choose ADMIN, normal profile updates cannot change security fields, and current account role/status are checked during JWT authentication.

30. Final Verification Summary

Feature	Verification Result
Application startup	BLOCKED - required secrets unavailable
MongoDB connection	BLOCKED
Passenger registration	PASS automated / BLOCKED live
Driver registration	PASS automated / BLOCKED live
Duplicate email validation	PASS automated / BLOCKED live
Input validation	PASS/PARTIAL by tested scenario
Password hashing	PASS inspection and unit test
Login	PASS automated / BLOCKED live
JWT issuance	PASS automated
JWT validation	PASS automated
Profile retrieval	PARTIAL automated / BLOCKED live
Profile update	PASS automated / BLOCKED live persistence
Account status management	PASS automated RBAC; live BLOCKED
RBAC	PASS automated
401 behavior	PASS automated
403 behavior	PASS automated
MongoDB persistence	BLOCKED live
Swagger UI	BLOCKED live
OpenAPI document	BLOCKED live
Automated tests	PASS - 23/23
Build	PASS
Secret handling	PASS

31. Viva Quick Reference

- **What is Account Service?** The security and identity boundary for RideLink passengers and drivers.
- **What data does it own?** Credentials, identity, role, profile, and status.
- **Why separate MongoDB?** Each microservice owns its persistence and can evolve independently.
- **Why no direct Driver Service access?** Direct database reads bypass authorization and couple schemas; use stable IDs and APIs.
- **Registration flow?** Validate DTO, normalize email, reject duplicates, BCrypt-hash password, assign server-controlled role/status, save, return safe DTO.
- **How BCrypt helps?** It salts and adaptively hashes passwords, making offline guessing expensive.
- **JWT flow?** Login signs time-limited claims; the filter validates signature/expiry and current ACTIVE role before authentication.
- **Authentication vs authorization?** Authentication identifies the caller; authorization determines permitted actions.
- **Roles?** PASSENGER and DRIVER are public identity types; ADMIN controls account status.
- **Statuses?** ACTIVE can authenticate; SUSPENDED and DISABLED cannot.

- **401?** Authentication is absent or invalid.
- **403?** An authenticated caller lacks permission, or a non-active account is denied login.
- **Why DTOs?** They validate input and prevent persistence/security fields from leaking or being mass-assigned.
- **Duplicate prevention?** Normalized pre-check plus a unique MongoDB email index.
- **Why never return passwordHash?** It is sensitive offline-guessing material and clients do not need it.
- **Secure Atlas configuration?** Environment variables, isolated database ownership, least privilege, and no committed credentials.
- **Tests performed?** 23 tests covering services, API validation, JWT expiry/claims, safe responses, and RBAC; Maven package build also passed.

Appendix A. PowerShell Smoke-Test Checklist

After supplying a test Atlas database and controlled admin account:

```
[ ] Start service and keep it running
[ ] Open Swagger and OpenAPI JSON
[ ] Register passenger; save passenger_id
[ ] Register driver; save driver_id
[ ] Repeat email; confirm 409
[ ] Login passenger; save token
[ ] GET own profile; confirm 200 and no passwordHash
[ ] GET without token; confirm 401
[ ] PUT name/phone; GET again and confirm persistence
[ ] PATCH status as passenger; confirm 403
[ ] PATCH status with controlled admin; confirm 200
[ ] Confirm SUSPENDED/DISABLED login denial
[ ] Restore test account to ACTIVE if authorized
[ ] Confirm Atlas records/index without exposing credentials or hashes
```

Appendix B. Evidence Classification

PASS means directly executed or covered by a passing automated test appropriate to the claim. PARTIAL means implementation or a subset was verified but a live dependency remained unavailable. BLOCKED means the requested live check could not execute because safe Atlas/JWT runtime configuration was not supplied. No other microservice was modified for this QA guide.